

Gestione delle date del calendario

Se avete la necessità di gestire le **date del calendario**, dovete partire dal fatto che **R** assume essere date del calendario quelle contenute in oggetti della classe "Date" ed espresse in uno specifico formato.

Vediamo di capire questa cosa con lo script che segue, copiatelo e incollatelo nella `Console di R` e premete ↵ Invio:

```
# GESTIONE DELLE DATE DEL CALENDARIO CON R - parte prima
#
# (1/5)
lemiedate <- c("13/05/1998", "16/12/1945", "23/09/2012") # i dati inseriti
lemiedate # il contenuto dell'oggetto
class(lemiedate) # la classe dell'oggetto
str(lemiedate) # la struttura dell'oggetto
#
# (2/5)
con_asDate <- as.Date(lemiedate, format="%d/%m/%Y") # i dati trasformati
con_asDate # il contenuto dell'oggetto
class(con_asDate) # la classe dell'oggetto
str(con_asDate) # la struttura dell'oggetto
#
# (3/5)
con_format <- format(con_asDate, "%d/%m/%Y") # i dati trasformati
con_format # il contenuto dell'oggetto
class(con_format) # la classe dell'oggetto
str(con_format) # la struttura dell'oggetto
#
# (4/5)
as.Date("23/09/2012", format="%d/%m/%Y") # dati in formato gg/mm/aaaa
#
as.Date("09/23/2012", format="%m/%d/%Y") # dati in formato mm/gg/aaaa
#
as.Date("2012/09/23", format="%Y/%m/%d") # dati in formato aaaa/mm/gg
#
as.Date("23 settembre 2012", format="%d %B %Y") # dati in formato gg mese aaaa con
spazio come separatore
#
as.Date("09-3-2012", format="%m-%d-%Y") # dati in formato mm-gg-aaaa con - come
separatore
#
# (5/5)
library(lubridate)
dmy(c("13/05/1998", "16/12/1945", "23/09/2012"))
#
```

Questo è il risultato della prima parte dello script:

```
> # (1/5)
> lemiedate <- c("13/05/1998", "16/12/1945", "23/09/2012") # i dati inseriti
> lemiedate # il contenuto dell'oggetto
[1] "13/05/1998" "16/12/1945" "23/09/2012"
> class(lemiedate) # la classe dell'oggetto
[1] "character"
```

```
> str(lemiedate) # la struttura dell'oggetto
chr [1:3] "13/05/1998" "16/12/1945" "23/09/2012"
> #
```

In pratica siamo partiti dal presupposto che

→ 13/05/1998

→ 16/12/1945

→ 23/09/2012

sono tre date del calendario che vogliamo gestire con **R**. Le abbiamo inserite nell'oggetto **lemiedate** con la funzione **c()**

```
> lemiedate <- c("13/05/1998", "16/12/1945", "23/09/2012") # i dati inseriti
```

le abbiamo ritrovate digitando il nome dell'oggetto

```
> lemiedate # il contenuto dell'oggetto
[1] "13/05/1998" "16/12/1945" "23/09/2012"
```

con la funzione **class()** che mostra la classe di un oggetto vediamo che **R** ha identificato le tre date come "character"

```
> class(lemiedate) # la classe dell'oggetto
[1] "character"
```

e con la funzione **str()** che mostra la struttura interna di un oggetto ne abbiamo riportato un ulteriore riepilogo sintetico

```
> str(lemiedate) # la struttura dell'oggetto
chr [1:3] "13/05/1998" "16/12/1945" "23/09/2012"
```

nel quale "chr" sta di nuovo per "character", la classe dell'oggetto.

Quindi inserendo i dati con la funzione **c()** quelle che per noi erano tre date sono state identificate in **R** come "character", cioè come semplici stringhe o in altre parole come tre sequenze di caratteri alfanumerici: in pratica invece delle date avremmo potuto inserire "ciao Giovanni", "domani farà bel tempo", "viva la pappa col pomodoro" o quant'altro.

Vediamo ora cosa accade con il successivo e secondo blocco di codice:

```
> # (2/5)
> con_asDate <- as.Date(lemiedate, format="%d/%m/%Y") # i dati trasformati
> con_asDate # il contenuto dell'oggetto
[1] "1998-05-13" "1945-12-16" "2012-09-23"
> class(con_asDate) # la classe dell'oggetto
[1] "Date"
> str(con_asDate) # la struttura dell'oggetto
Date[1:3], format: "1998-05-13" "1945-12-16" "2012-09-23"
> #
```

Questa volta impieghiamo la funzione **as.Date()** per inserire **lemiedate** (l'oggetto della classe "character" contenente tre stringhe alfanumeriche) in un nuovo oggetto denominato **con_asDate** (giusto per ricordarci come lo abbiamo generato)

```
> con_asDate <- as.Date(lemiedate, format="%d/%m/%Y") # i dati trasformati
```

Dopo avere ritrovato quello che abbiamo inserito digitando il nome dell'oggetto

```
> con_asDate # il contenuto dell'oggetto
[1] "1998-05-13" "1945-12-16" "2012-09-23"
```

con la funzione **class()**

```
> class(con_asDate) # la classe dell'oggetto
[1] "Date"
```

e con la funzione **str()**

```
> str(con_asDate) # la struttura dell'oggetto
Date[1:3], format: "1998-05-13" "1945-12-16" "2012-09-23"
> #
```

vediamo che **con_asDate** è un oggetto della classe "Date" nel quale in **R** per definizione sono contenute date del calendario

L'argomento **format="...."** impiegato nella funzione **as.Date()** specifica sia l'ordine sia il formato dei dati in ingresso con i seguenti codici:

Codice	Valore
%d	Giorno del mese (numero decimale)
%m	Mese (numero decimale)
%b	Mese (abbreviato)
%B	Mese (nome esteso)
%y	Anno (2 cifre)
%Y	Anno (4 cifre)

Se ora nella Console di R digitate **help(as.Date)** si apre una pagina web con la definizione della funzione in questione, che è definita come *"la funzione che converte le rappresentazioni [di date] sotto forma di caratteri [stringhe di testo] in oggetti della classe "Date" che rappresentano date di calendario"*.

Come certamente avrete notato una volta importate in **R** con la funzione **as.Date()** le date sono state convertite nel formato *aaaa-mm-gg*. A questo punto la domanda è d'obbligo: e se volessi tornare a convertirle nel formato *gg/mm/aaaa*? La cosa è possibile, e vediamo come con la terza parte di codice:

```
> # (3/5)
> con_format <- format(con_asDate, "%d/%m/%Y") # i dati trasformati
> con_format # il contenuto dell'oggetto
[1] "13/05/1998" "16/12/1945" "23/09/2012"
> class(con_format) # la classe dell'oggetto
[1] "character"
> str(con_format) # la struttura dell'oggetto
chr [1:3] "13/05/1998" "16/12/1945" "23/09/2012"
> #
```

Con la funzione **format()** - da non confondere con l'argomento **format=** della funzione **as.Date()** - possiamo convertire le date dal formato *aaaa-mm-gg* nel quale erano nell'oggetto **con_asDate** al formato *gg/mm/aaaa* specificato con l'argomento **"%d/%m/%Y"**

```
> con_format <- format(con_asDate, "%d/%m/%Y") # i dati trasformati
```

Questo può essere utile ad esempio se vogliamo visualizzare o stampare le date in questo formato. Ma in questo modo le date per **R** non sono più tali. Se esaminiamo il nuovo oggetto **con_format**

```
> con_format # il contenuto dell'oggetto
[1] "13/05/1998" "16/12/1945" "23/09/2012"
```

con la funzione **class()**

```
> class(con_format) # la classe dell'oggetto
[1] "character"
```

e con la funzione **str()**

```
> str(con_format) # la struttura dell'oggetto
chr [1:3] "13/05/1998" "16/12/1945" "23/09/2012"
> #
```

vediamo che si tratta di un oggetto di classe "character": imponendo il formato *gg/mm/aaaa* le date tornano ad essere nuovamente delle stringhe ovvero generiche sequenze di caratteri alfanumerici esattamente come quelle da cui eravamo partiti.

Quindi concludendo: le date del calendario in **R** sono per definizione quelle contenute in un oggetto della classe "Date" ed espresse nel formato *aaaa-mm-gg*.

Vediamo ora alcuni altri esempi di date importate in **R** mediante la funzione **as.Date()**:

```
> # (4/5)
> as.Date("23/09/2012", format="%d/%m/%Y") # dati in formato gg/mm/aaaa
[1] "2012-09-23"
> #
> as.Date("09/23/2012", format="%m/%d/%Y") # dati in formato mm/gg/aaaa
[1] "2012-09-23"
> #
> as.Date("2012/09/23", format="%Y/%m/%d") # dati in formato aaaa/mm/gg
[1] "2012-09-23"
> #
> as.Date("23 settembre 2012", format="%d %B %Y") # dati in formato gg mese aaaa con
spazio come separatore
[1] "2012-09-23"
> #
> as.Date("09-3-2012", format="%m-%d-%Y") # dati in formato mm-gg-aaaa con - come
separatore
[1] "2012-09-03"
> #
```

I risultati confermano che se specificate opportunamente l'argomento **format=** dei dati in ingresso potete rappresentare sotto forma di stringhe alfanumeriche le vostre date predisposte:

- nel formato *gg/mm/aaaa*
- nel formato *mm/gg/aaaa*
- nel formato *aaaa/gg/mm*
- nel formato *gg mese aaaa* impiegando lo spazio come separatore
- nel formato *mm-gg-aaaa* impiegando come separatore il tratto a metà altezza -

ovvero in altri modi purché sia correttamente e completamente specificato il formato dei dati in ingresso.

Se installate il pacchetto **lubridate** potete infine fruire di una sintassi più concisa come nell'esempio che segue, che porta ovviamente allo stesso risultato

```
> # (5/5)
> library(lubridate)
> dmy(c("13/05/1998", "16/12/1945", "23/09/2012"))
[1] "1998-05-13" "1945-12-16" "2012-09-23"
```

Vediamo ora cosa accade nel caso di **date importate da un file** in formato Excel (`.xls` o `.xlsx`) o, meglio ancora, in formato `.csv` (il formato dei dati da importare consigliato da **R**).

Andate alla [pagina Dati](#) nella quale trovate diverse opzioni per scaricare i file di dati, quindi scaricate e copiate nella cartella `C:\Rdati\` il file `PA.xls` e il file `PA.csv`

Nei file sono riportati i valori della pressione arteriosa sistolica e diastolica e della frequenza cardiaca registrati alle date del calendario riportate nella variabile "Data" (prima colonna).

	A	B	C	D
1	Data	Sistolica	Diastolica	Frequenza
2	07/05/2021	112	64	50
3	08/05/2021	118	69	49
4	08/05/2021	101	57	62
5	09/05/2021	111	65	56
6	09/05/2021	117	67	60
7	10/05/2021	110	69	59
8	10/05/2021	109	63	53
9	16/05/2021	115	66	49
10	17/09/2021	121	65	48
11	17/09/2021	92	58	63
12	17/09/2021	131	79	68
13	17/09/2021	116	68	57
14	29/09/2021	114	69	56

Se ancora non l'avete fatto, scaricate dal **CRAN** e installate il pacchetto **xlsx**, quindi copiate e incollate nella Console di R questo script e premete ↵ Invio:

```

# GESTIONE DELLE DATE DEL CALENDARIO CON R - parte seconda
#
# importare le date da un file .xls
#
require(xlsx) # carica il pacchetto xlsx
PA_xls <- read.xlsx("C:/Rdati/PA.xls", sheetName="M") # importa i dati del file e del foglio
specificati
PA_xls # mostra la tabella che contiene i dati
str(PA_xls) # la variabile Data viene riconosciuta come una data del calendario (classe "Date")
#
# importare le date da un file .csv
#
PA_csv <- read.table("C:/Rdati/PA.csv", header=TRUE, sep=",") # carica i dati del file
specificato
PA_csv # mostra la tabella che contiene i dati
str(PA_csv) # la variabile Data viene riconosciuta come una semplice stringa di testo (classe
"character")
#
PA_csv$Data <- as.Date(PA_csv$Data, format="%d/%m/%Y") # la stringa Data viene
convertita con as.Date
str(PA_csv) # la variabile Data viene ora riconosciuta come una data del calendario
#
# elaborare le date (esempi)
#
Sys.Date()
PA_xls$Data[13] - PA_xls$Data[1]
PA_xls$Data[13] + 32
weekdays(PA_csv$Data[1])
#

```

Dopo avere caricato il pacchetto **xlsx**

```

> require(xlsx) # carica il pacchetto xlsx
Caricamento del pacchetto richiesto: xlsx

```

i dati sono importati dal foglio **M** del file **PA.xls**

```

> PA_xls <- read.xlsx("C:/Rdati/PA.xls", sheetName="M") # importa i dati del file e
del foglio specificati

```

e richiamando il nome dell'oggetto (**PA_xls**) nel quale sono stati importati vediamo che i dati sono stati importati correttamente

```

> PA_xls # mostra la tabella che contiene i dati

```

	Data	Sistolica	Diastolica	Frequenza
1	2021-05-07	112	64	50
2	2021-05-08	118	69	49
3	2021-05-08	101	57	62
4	2021-05-09	111	65	56
5	2021-05-09	117	67	60
6	2021-05-10	110	69	59
7	2021-05-10	109	63	53
8	2021-05-16	115	66	49
9	2021-09-17	121	65	48
10	2021-09-17	92	58	63

11	2021-09-17	131	79	68
12	2021-09-17	116	68	57
13	2021-09-29	114	69	56

La funzione **str()** ci conferma che anche la variabile `Data` è stata correttamente identificata come `"Date"` quindi il pacchetto **xlsx** ha trasferito i dati della prima colonna in modo tale che **R** li ha riconosciuti e li riporta come date del calendario.

```
> str(PA_xls) # la variabile Data viene riconosciuta come una data del calendario
(classe "Date")
'data.frame': 13 obs. of 4 variables:
 $ Data      : Date, format: "2021-05-07" "2021-05-08" ...
 $ Sistolica : num  112 118 101 111 117 110 109 115 121 92 ...
 $ Diastolica: num  64 69 57 65 67 69 63 66 65 58 ...
 $ Frequenza : num  50 49 62 56 60 59 53 49 48 63 ...
> #
```

Vediamo ora cosa accade quando si importano i dati da un file `.csv`

Dopo avere importato i dati nell'oggetto **PA_csv**

```
> # importare le date da un file .csv
> #
> PA_csv <- read.table("C:/Rdati/PA.csv", header=TRUE, sep=",") # carica i dati del
file specificato
```

e averli visualizzati richiamando il nome dell'oggetto nel quale sono stati importati

```
> PA_csv # mostra la tabella che contiene i dati
      Data Sistolica Diastolica Frequenza
1 07/05/2021      112         64         50
2 08/05/2021      118         69         49
3 08/05/2021      101         57         62
4 09/05/2021      111         65         56
5 09/05/2021      117         67         60
6 10/05/2021      110         69         59
7 10/05/2021      109         63         53
8 16/05/2021      115         66         49
9 17/09/2021      121         65         48
10 17/09/2021       92         58         63
11 17/09/2021      131         79         68
12 17/09/2021      116         68         57
13 29/09/2021      114         69         56
```

con la funzione **str()** vediamo che questa volta la variabile `Data` è stata identificata come una stringa / generica sequenza di caratteri alfanumerici (`chr`)

```
> str(PA_csv) # la variabile Data viene riconosciuta come una semplice stringa di
testo (classe "character")
'data.frame': 13 obs. of 4 variables:
 $ Data      : chr  "07/05/2021" "08/05/2021" "08/05/2021" "09/05/2021" ...
 $ Sistolica : int  112 118 101 111 117 110 109 115 121 92 ...
 $ Diastolica: int  64 69 57 65 67 69 63 66 65 58 ...
 $ Frequenza : int  50 49 62 56 60 59 53 49 48 63 ...
> #
```

Questo accade perché nel file `PA.csv` che aperto con un editor di testo così ci appare

Data,Sistolica,Diastolica,Frequenza

```
07/05/2021,112,64,50
08/05/2021,118,69,49
08/05/2021,101,57,62
09/05/2021,111,65,56
09/05/2021,117,67,60
10/05/2021,110,69,59
10/05/2021,109,63,53
16/05/2021,115,66,49
17/09/2021,121,65,48
17/09/2021,92,58,63
17/09/2021,131,79,68
17/09/2021,116,68,57
29/09/2021,114,69,56
```

non è riportata alcuna informazione che consenta ad **R** di identificare la variabile `Data` come una data del calendario (nel file `.xls` questa informazione c'è, anche se non potete visualizzarla trattandosi di un file binario).

Nel caso di dati importati da un file `.csv` - che di fatto è un semplice file di testo - è pertanto necessario impiegare la funzione **`as.Date()`** per convertire le date/stringhe in ingresso in date del calendario:

```
> PA_csv$Data <- as.Date(PA_csv$Data, format="%d/%m/%Y") # la stringa Data viene
convertita con as.Date
> str(PA_csv) # la variabile Data viene ora riconosciuta come una data del calendario
'data.frame': 13 obs. of 4 variables:
 $ Data      : Date, format: "2021-05-07" "2021-05-08" ...
 $ Sistolica : int  112 118 101 111 117 110 109 115 121 92 ...
 $ Diastolica: int   64 69 57 65 67 69 63 66 65 58 ...
 $ Frequenza : int   50 49 62 56 60 59 53 49 48 63 ...
> #
```

Questa conversione è necessaria se vogliamo che **R** riconosca le operazioni che è possibile eseguire su questi oggetti, che saranno, appunto, solamente le operazioni applicabili alle date del calendario.

A questo punto la domanda: ma perché fare tanta fatica per arrivare fin qui? La risposta la trovate a conclusione dello script, laddove sono riportate alcune delle possibilità offerte da **R** nella gestione delle date che prima o poi possono tornare utili, come ad esempio:

→ la data attuale presente sul sistema

```
> Sys.Date()
[1] "2021-11-29"
```

→ la differenza in giorni tra due date

```
> PA_xls$Data[13] - PA_xls$Data[1]
Time difference of 145 days
```

→ la data corrispondente a una specifica differenza in giorni rispetto a una data di riferimento

```
> PA_xls$Data[13] + 32
[1] "2021-10-31"
```


→ il giorno della settimana corrispondente ad una specifica data

```
> weekdays(PA_csv$Data[1])  
[1] "venerdì"  
> #
```

In un prossimo post vedremo come gestire contemporaneamente data e ora.
